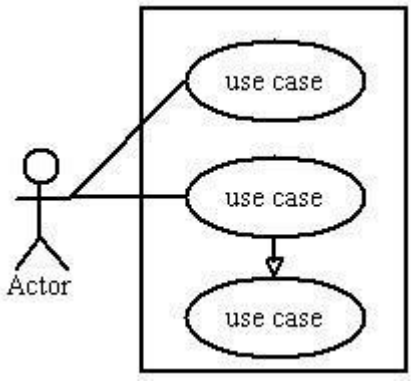
Final Project CS619 VIVA Preparation Notes

We will Also Have a Project Discussion If needed then Contact with me

0300-8959571

Use Case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

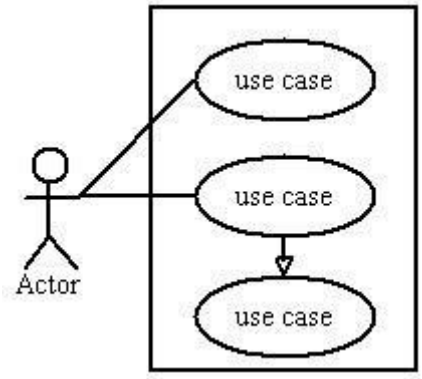


Basic Use Case Diagram Symbols and Notations

System

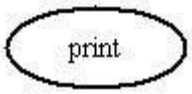
Draw your system's boundaries using a rectangle that contains use cases. Place actors outside the system's boundaries.

System is used to define the scope of the use case and drawn as a rectangle. This an optional element but useful when your visualizing large systems. For example you can create all the use cases and then use the system object to define the scope covered by your project. Or you can even use it to show the different areas covered in different releases.



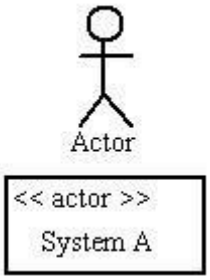
Use Case

Draw use cases using ovals. Label with ovals with verbs that represent the system's functions.



Actors

Actors are the users of a system. When one system is the actor of another system, label the actor system with the actor stereotype.

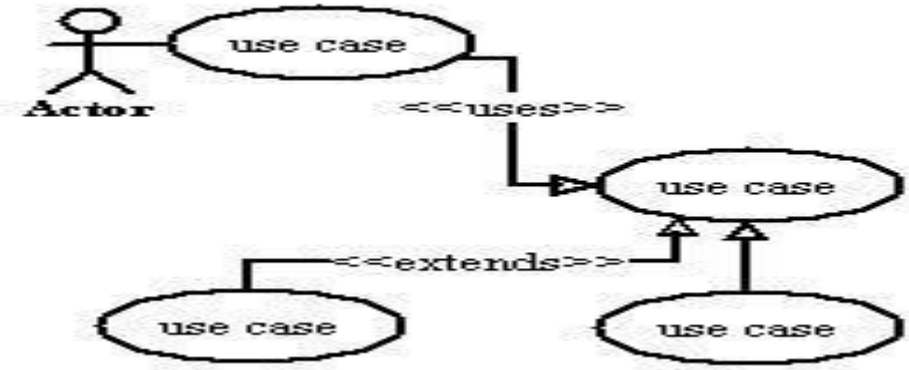


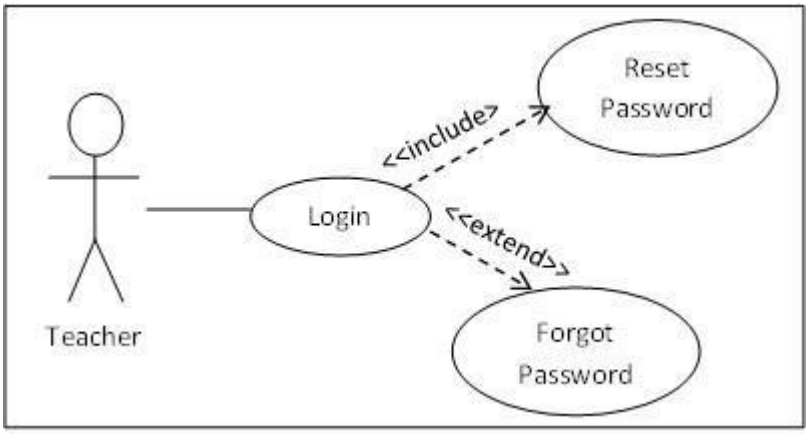
Relationships

Illustrate relationships between an actor and a use case with a simple line. For relationships among use cases, use arrows labeled either "uses" or "extends" or "includes

A "uses" relationship indicates that one use case is needed by another in order to perform a task.
An "extends" relationship indicates alternative options under a certain use case.

An include relationship is a relationship between two use cases It indicates that the use case to which the arrow points is included in the use case on the other side of the arrow. This makes it possible to reuse a use case in another use case.





The following is a sample use case diagram representing the order management system. So if we look into the diagram then we will find three use cases (Order, Special Order and Normal Order) and one actor which is customer.

The Special Order and Normal Order use cases are extended from Order use case. So they have extends relationship. Another important point is to identify the system boundary which is shown in the picture. The actor Customer lies outside the system as it is an external user of the system.

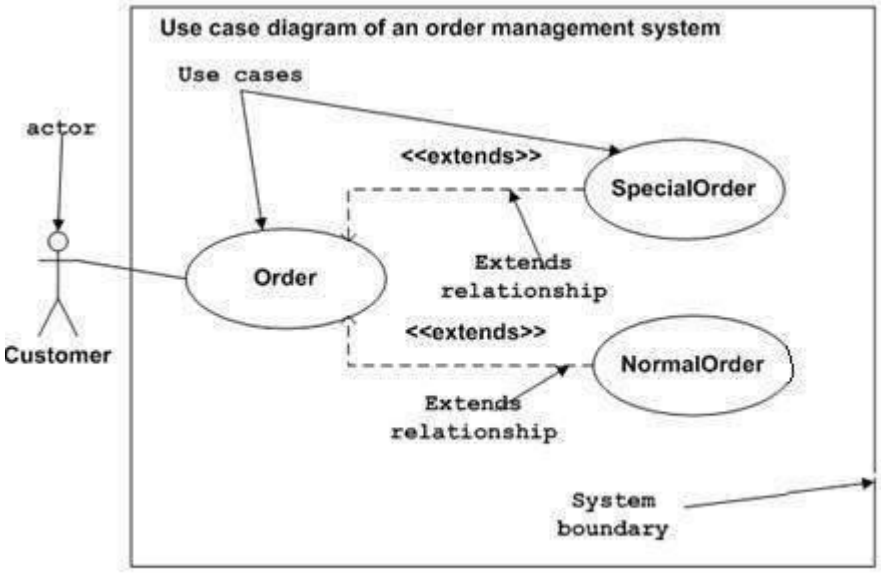


Figure: Sample Use Case diagram

Entity Relationship Diagram (ERD)

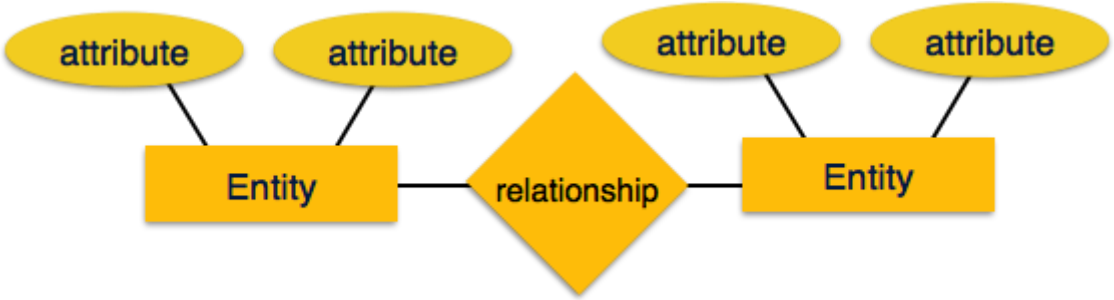
An entity-relationship diagram—otherwise known as an ERD—is a data modeling technique that creates an illustration of an information system's entities and the relationships between those entities.

3 ingredients of entity-relationship diagram

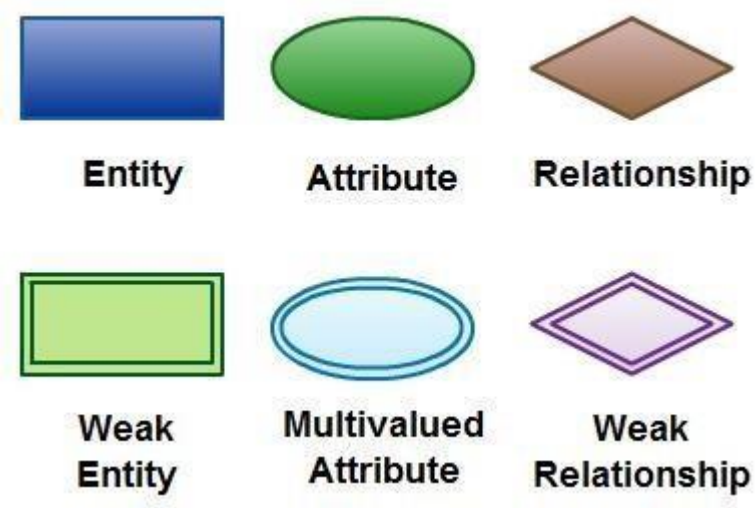
Entities, which represent people, places, items, events, or concepts.

Attributes, which represent properties or descriptive qualities of an entity. These are also known as data elements.

Relationships, which represent the link between different entities.



ER Diagram Symbols and Notations



Cardinality and Modality

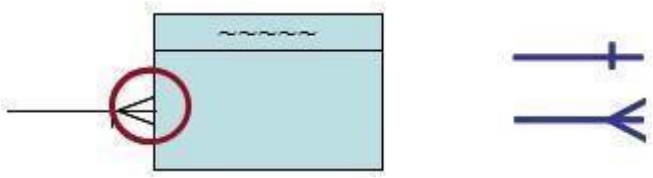
Cardinality and Modality work together to define the relationship.

- Cardinality indicates the maximum number of times an instance in one entity can be associated with instances in the related entity.
- Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity.

Thus, Modality is also called participation because it denotes whether or not an instance of an entity MUST participate in the relationship.

Cardinality and Modality are both shown on the relationship line by symbols. We will go over each of the symbols and how to interpret them.

Cardinality

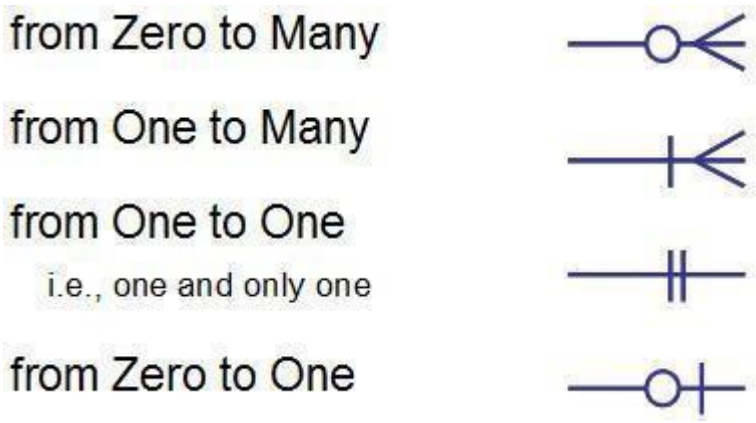


Cardinality indicates the maximum number of times an instance of one entity can be associated with instances in the related entity. Cardinality can have the values of one or many, no more detail than that. It is either one or more than one. On the relationship line, the cardinality is the closest to the entity box. The cardinality symbol in the diagram on the slide is in the red circle. Cardinality is indicated at BOTH ends of the relationship line, so there is a left to right cardinality and a right to left cardinality. **Modality**



Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity. Modality can have the values of zero or one, two or three are not allowed. The modality symbol is located next to the cardinality symbol, on the inside, i.e., NOT next to the entity box. A modality of one is denoted by a straight vertical line and a modality of zero is denoted by a circle. Like cardinality, modality is indicated at both ends of the relationship.

Reading Modality and Cardinality



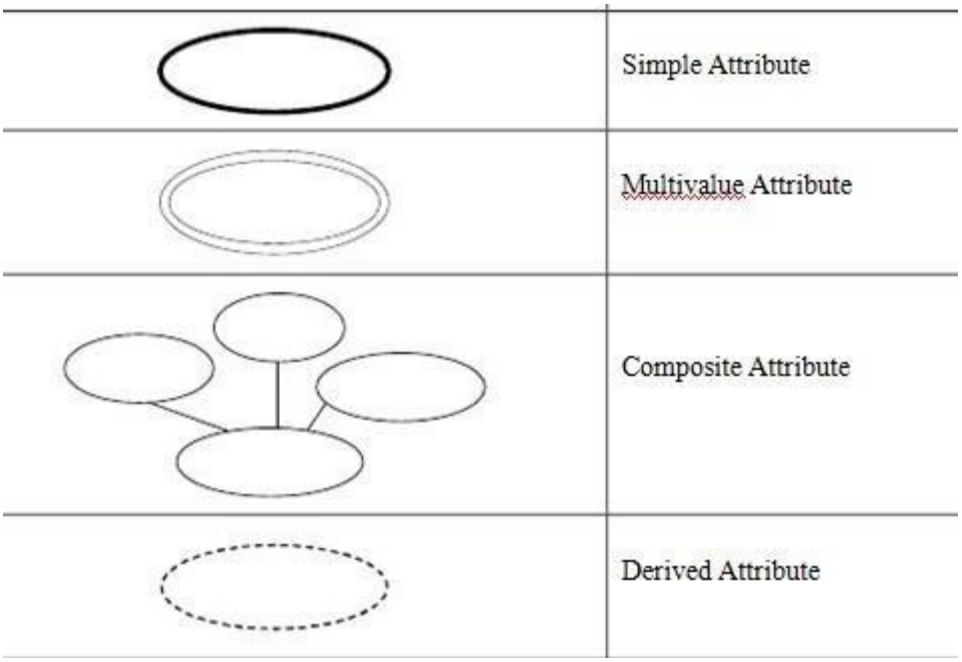
Types of Attributes · Simple attribute − Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.

- Composite attribute − Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.

- Derived attribute − Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average_salary in a department should not be saved

directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.

- Single-value attribute − Single-value attributes contain single value. For example − Social_Security_Number.
- Multi-value attribute − Multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.



Types of Entity –

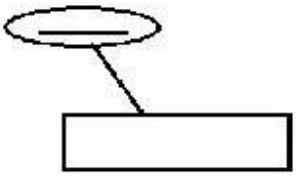
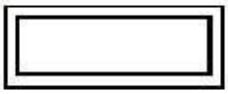
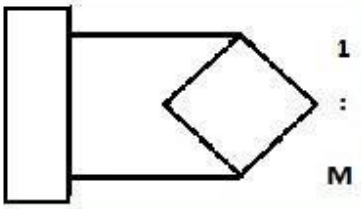
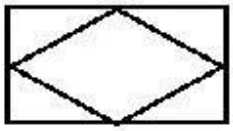
Strong Entity Types

Recursive Entity Types

Weak Entity Types

Composite Entity Types or Associative Entity Types

Notations Of different Entity Type in ER Diagram

	Entity
	Strong Entity Type
	Weak Entity Type
	Recursive Entity Type
	Composite Entity Type (or) Associative Entity

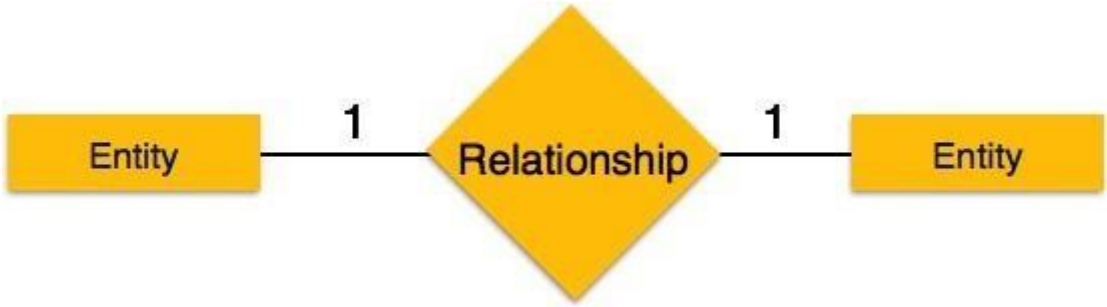
Relationship

Relationships are represented by diamond-shaped box. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.

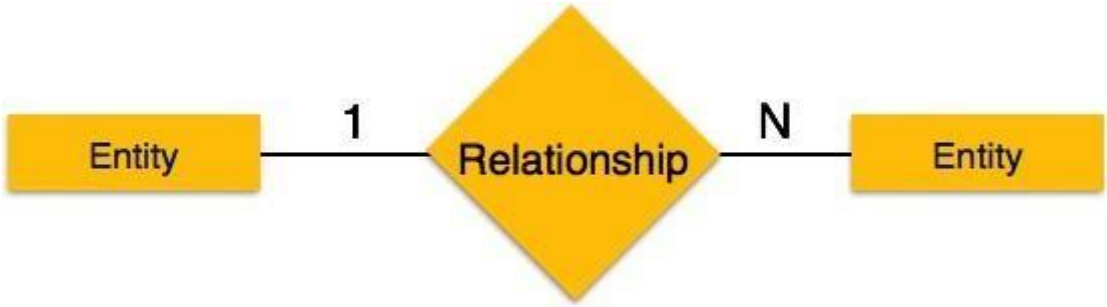
Binary Relationship and Cardinality

A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

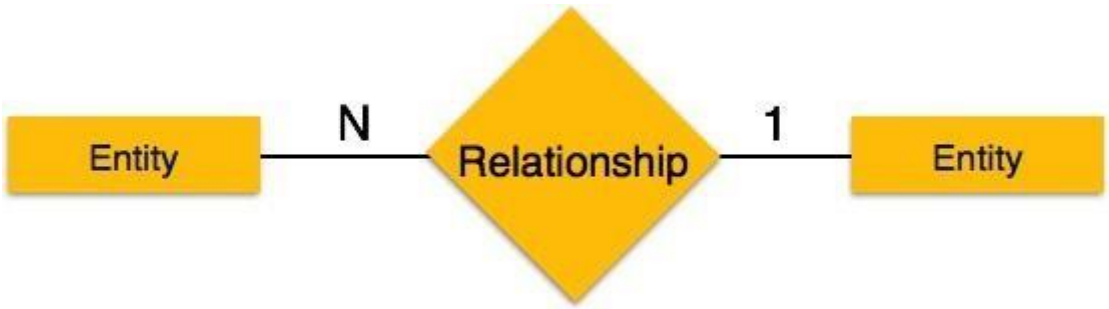
- One-to-one − When only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one instance of each entity should be associated with the relationship. It depicts onetoone relationship.



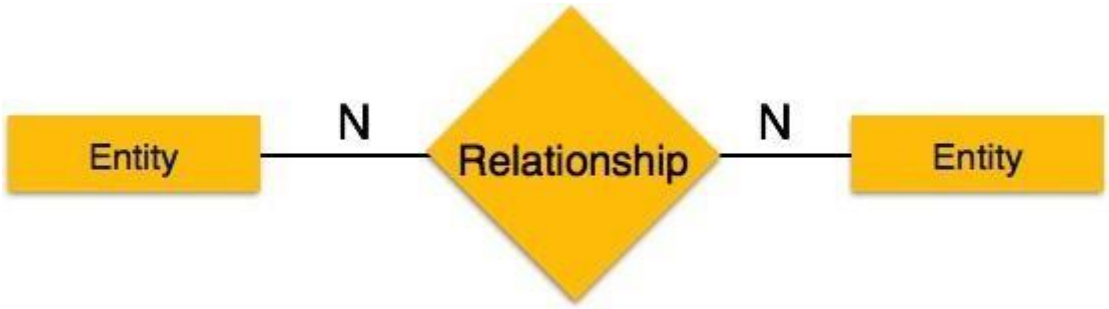
- One-to-many − When more than one instance of an entity is associated with a relationship, it is marked as '1:N'. The following image reflects that only one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts one-tomany relationship.



- Many-to-one − When more than one instance of entity is associated with the relationship, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts manyto-one relationship.



- Many-to-many − The following image reflects that more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts many-to-many relationship.



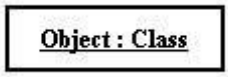
Sequence Diagrams

The Sequence Diagram shows how the objects interact with others in a particular scenario of a use case.

Basic Sequence Diagram Symbols and Notations

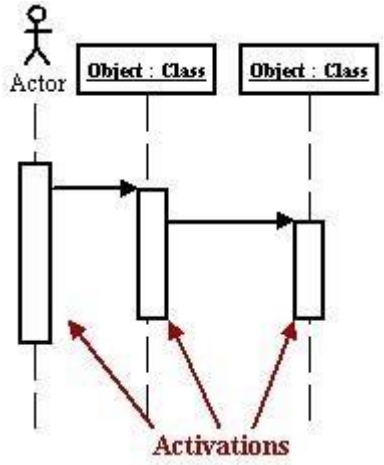
Class roles

Class roles describe the way an object will behave in context. Use the UML object symbol to illustrate class roles, but don't list object attributes.



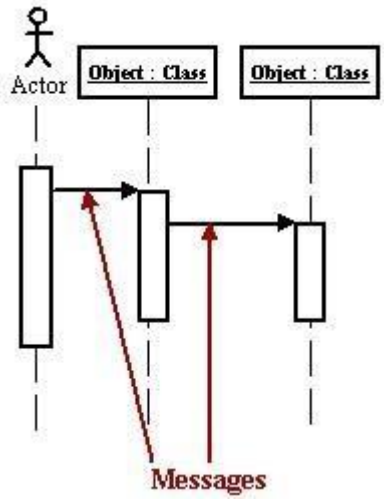
Activation

Activation boxes represent the time an object needs to complete a task.



Messages

Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.



Various message types for Sequence and Collaboration diagrams

Types of Messages in Sequence Diagrams

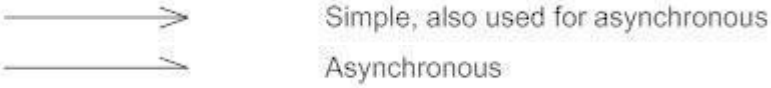
Synchronous Message

A synchronous message requires a response before the interaction can continue. It's usually drawn using a line with a solid arrowhead pointing from one object to another.



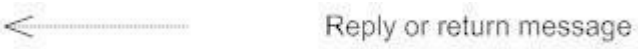
Asynchronous Message

Asynchronous messages don't need a reply for interaction to continue. Like synchronous messages, they are drawn with an arrow connecting two lifelines; however, the arrowhead is usually open and there's no return message depicted.



Reply or Return Message

A reply message is drawn with a dotted line and an open arrowhead pointing back to the original lifeline.

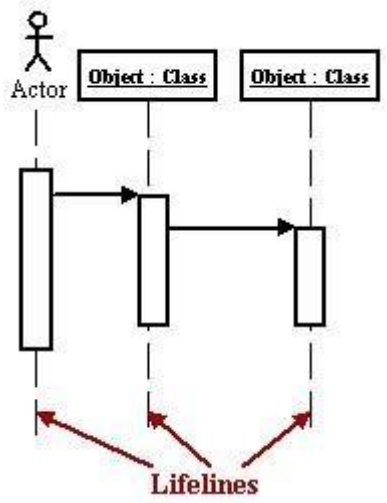


Self Message

A message an object sends to itself, usually shown as a U shaped arrow pointing back to itself.

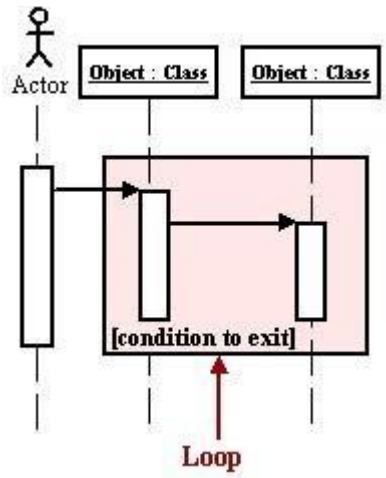
Lifelines

Lifelines are vertical dashed lines that indicate the object's presence over time.



Loops

A repetition or loop within a sequence diagram is depicted as a rectangle. Place the condition for exiting the loop at the bottom left corner in square brackets [ ].



Basic Sequence Diagram Symbols and Notations

Class Roles or Participants

Class roles describe the way an object will behave in context. Use the UML object symbol to illustrate class roles, but don't list object attributes.



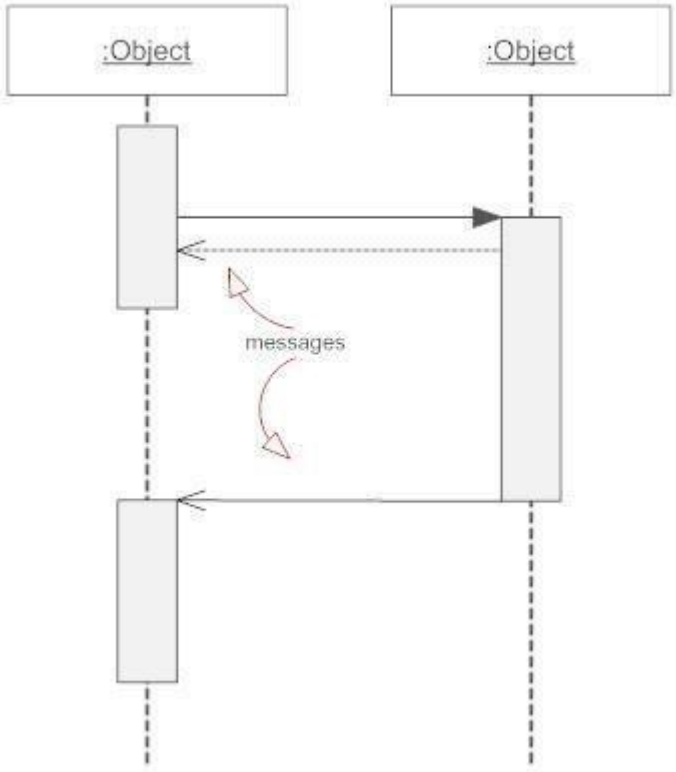
Activation or Execution Occurrence

Activation boxes represent the time an object needs to complete a task. When an object is busy executing a process or waiting for a reply message, use a thin gray rectangle placed vertically on its lifeline.



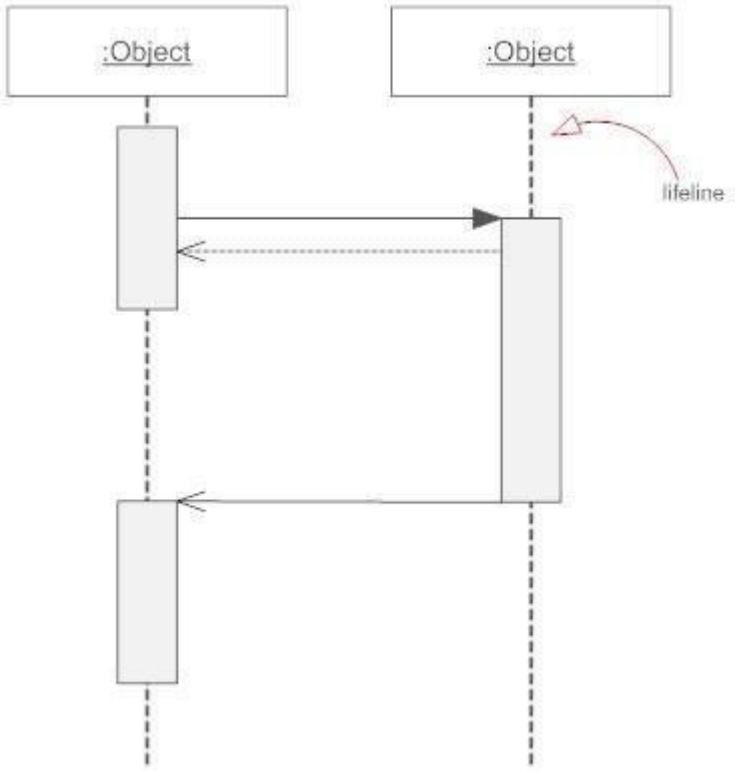
Messages

Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks. For message types, see below.



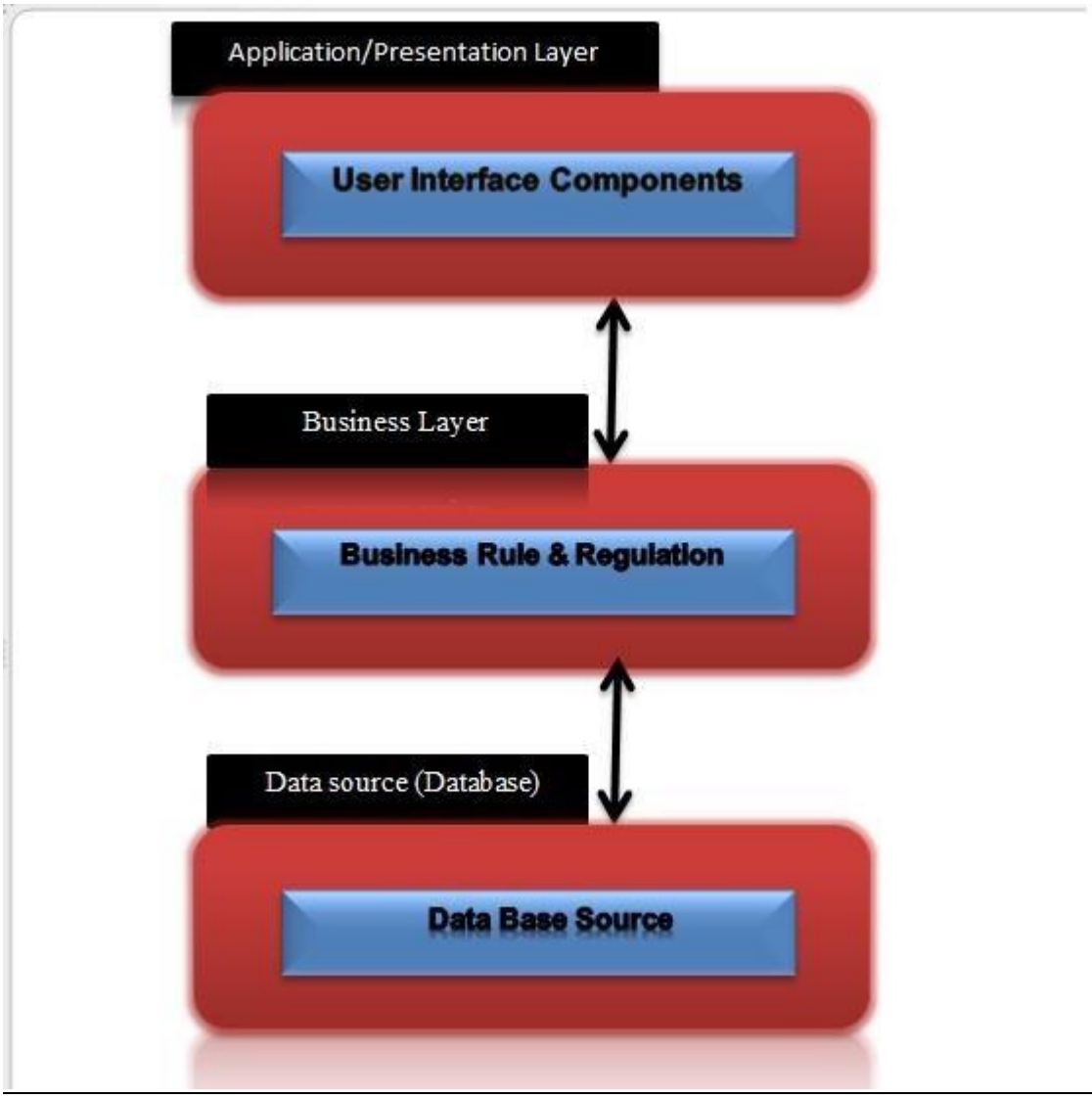
Lifelines

Lifelines are vertical dashed lines that indicate the object's presence over time.



Architecture Design Diagram

The architecture adopted is Three-tier web architecture. The architecture consists of three layers which are:



---

1. Presentation tier

Front End is designed in programming language like PHP, C# or Java etc...

2. Application tier

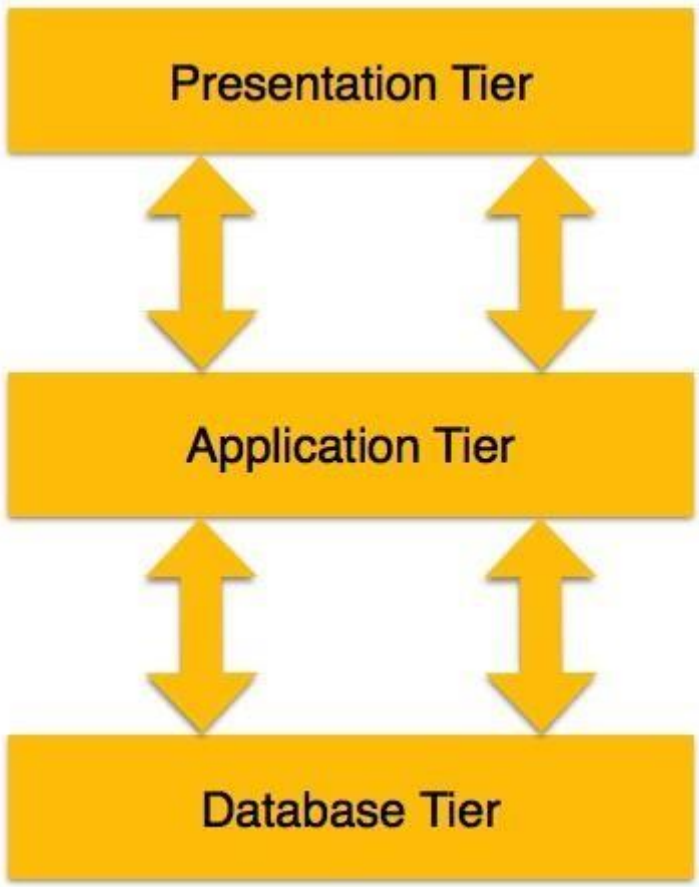
The middle dynamic content processing and generation level server layer is created in PHP, C# or Java etc...

3. Data tier

SQL or MySQL is used as back-end database server.

3-tier Architecture

A 3-tier architecture separates its tiers from each other based on the complexity of the users and how they use the data present in the database. It is the most widely used architecture to design a DBMS.



- **Database (Data) Tier** − At this tier, the database resides along with its query processing languages. We also have the relations that define the data and their constraints at this level.

- **Application (Middle) Tier** − At this tier reside the application server and the programs that access the database.

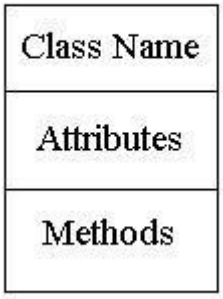
For a user, this application tier presents an abstracted view of the database. End-users are unaware of any existence of the database beyond the application. At the other end, the database tier is not aware of any other user beyond the application tier. Hence, the application layer sits in the middle and acts as a mediator between the end-user and the database.

- **User (Presentation) Tier** − End-users operate on this tier and they know nothing about any existence of the database beyond this layer. At this layer, multiple views of the database can be provided by the application. All views are generated by applications that reside in the application tier.

Class Diagram

Class diagrams are used to represents real life objects in our design by capturing its data methods and relationships of these methods with each other. These classes are basic building blocks of our object oriented system. Class diagrams are represented with white boxes which consist of three parts. Each of these parts has unique characteristics.

Syntax

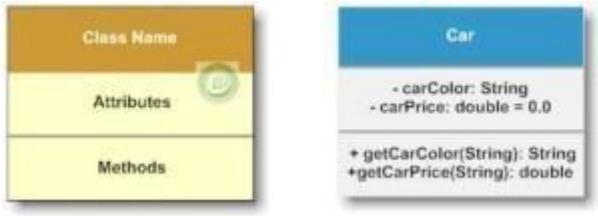


The details of these parts are as follow.

1. The upper part contains the name of the class

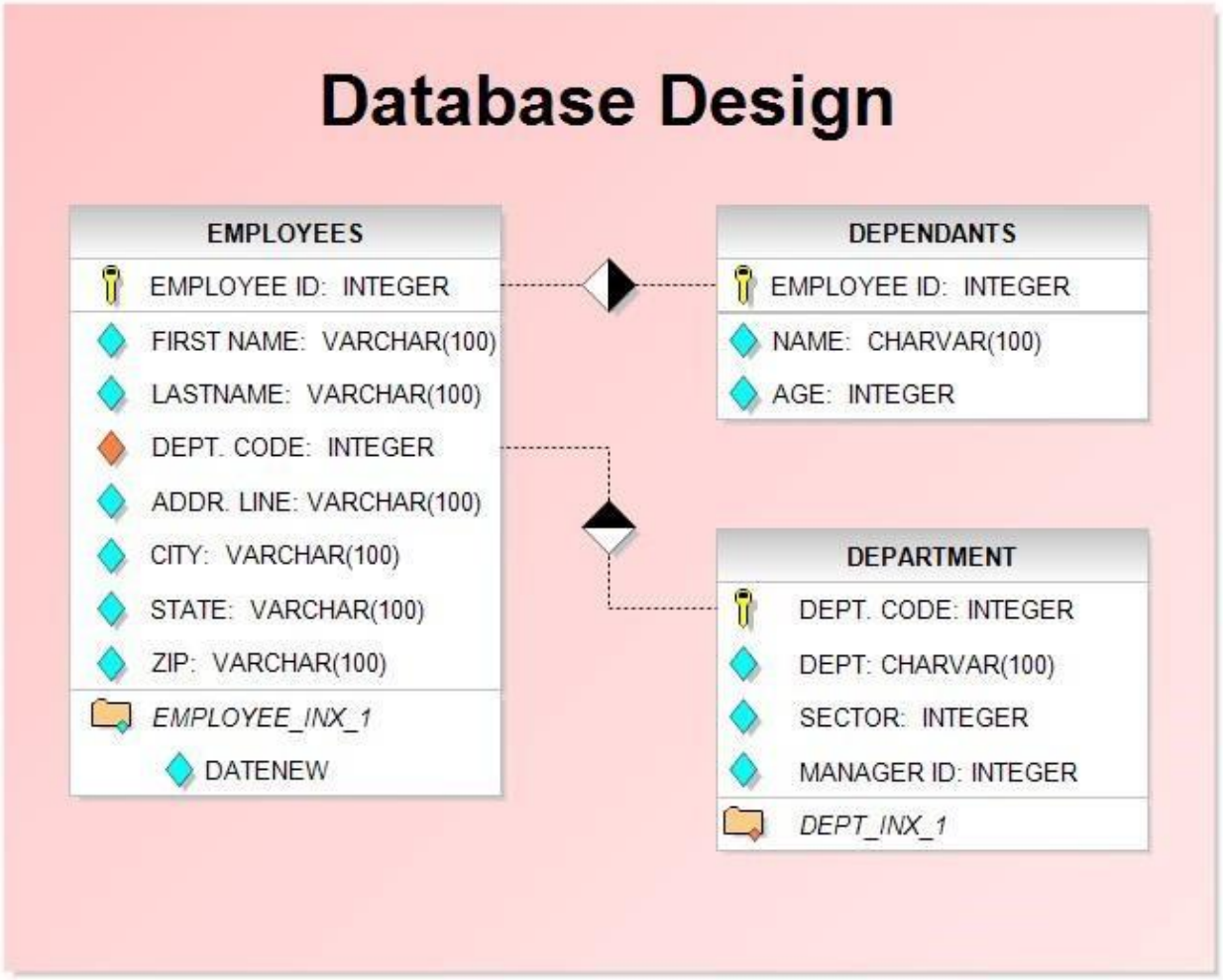
2. The next part contains the attributes of the class.
3. The third part contains the methods and operations on the class.

Class Diagram Example



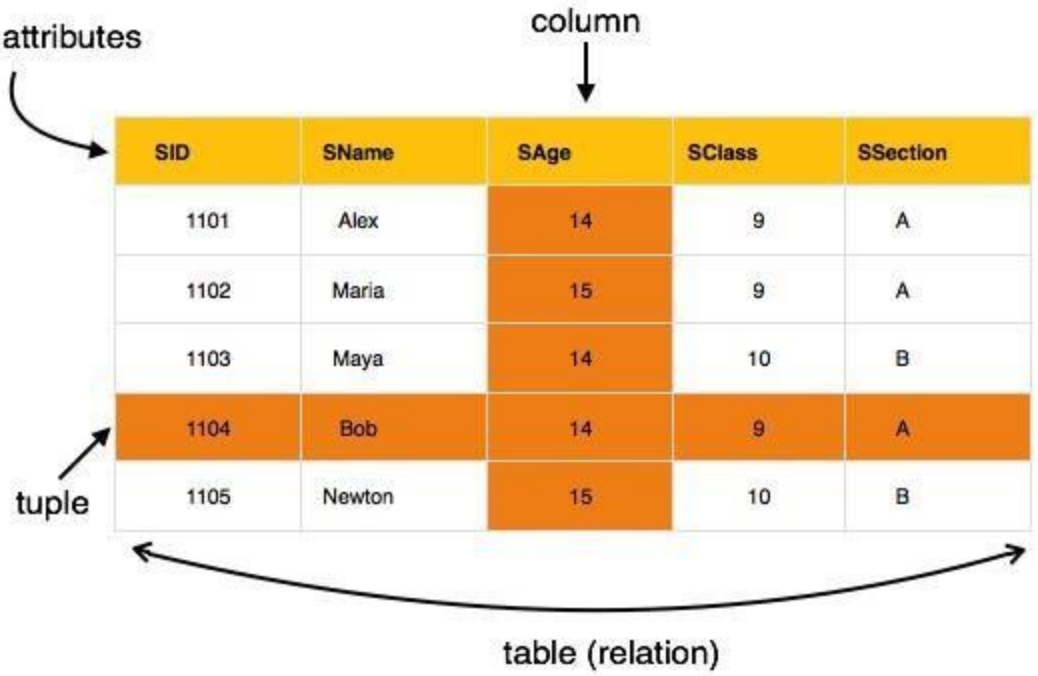
Database Design

In our daily life every object has some data associated with it. This data can be managed through data base design. Data large data can be represented by this data base design. Data base design can be represented in the form of physical, conceptual and logical.



Relational Model

The most popular data model in DBMS is the Relational Model. It is more scientific a model than others. This model is based on first-order predicate logic and defines a table as an n-ary relation.



The main highlights of this model are −

- Data is stored in tables called relations.
- Relations can be normalized.
- In normalized relations, values saved are atomic values.
- Each row in a relation contains a unique value.
- Each column in a relation contains values from a same domain.

Relation Data Model Concepts Tables − In relational data model, relations are saved in the format of Tables. This format stores the relation among entities. A table has rows and columns, where rows represents records and columns represent the attributes.

Tuple − A single row of a table, which contains a single record for that relation is called a tuple.

Relation instance − A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.

Relation schema − A relation schema describes the relation name (table name), attributes, and their names.

Relation key − Each row has one or more attributes, known as relation key, which can identify the row in the relation (table) uniquely.

Attribute domain − Every attribute has some pre-defined value scope, known as attribute domain.